

3.1.2 由秩到位置

改用以上动态存储策略之后，在提高动态操作效率的同时，却又不得不舍弃原静态存储策略中循秩访问的方式，从而造成静态操作性能的下降。

以采用动态存储策略的线性结构（比如链表）为例。尽管按照逻辑次序，每个数据元素依然具有秩这一指标，但为了访问秩为 r 的元素，我们只能顺着相邻元素之间的指针，从某一端出发逐个扫描各元素，经过 r 步迭代后才能确定该元素的物理存储位置。这意味着，原先只需 $O(1)$ 时间的静态操作，此时的复杂度也将线性正比于被访问元素的秩，在最坏情况下等于元素总数 n ；即便在各元素被访问概率相等的情况下，平均而言也需要 $O(n)$ 时间。

对数据结构的访问方式，应与其存储策略相一致。此时，既然继续沿用循秩访问的方式已非上策，就应更多地习惯于通过位置，来指代并访问动态存储结构中的数据元素。与向量中秩的地位与功能类似，列表中的位置也是指代各数据元素的一个标识性指标，借助它可以便捷地（比如在常数时间内）得到元素的物理存储地址。各元素的位置，通常可表示和实现为联接于元素之间的指针或引用。因此，基于此类结构设计算法时，应更多地借助逻辑上相邻元素之间的位置索引，以实现目标元素的快速定位和访问，并进而提高算法的整体效率。

3.1.3 列表

与向量一样，列表也是由具有线性逻辑次序的一组元素构成的集合：

$$L = \{ a_0, a_1, \dots, a_{n-1} \}$$

列表是链表结构的一般化推广，其中的元素称作节点（node），分别由特定的位置或链接指代。与向量一样，在元素之间，也可定义前驱、直接前驱，以及后继、直接后继等关系；相对于任意元素，也有定义对应的前缀、后缀等子集。

§ 3.2 接口

如上所述，作为列表的基本组成单位，列表节点除需保存对应的数据项，还应记录其前驱和后继的位置，故需将这些信息及相关操作组成列表节点对象，然后参与列表的构建。

本节将给出列表节点类与列表类的接口模板类描述，稍后逐一讲解各接口的具体实现。

3.2.1 列表节点

■ ADT接口

作为一种抽象数据类型，列表节点对象应支持以下操作接口。

表3.1 列表节点ADT支持的操作接口

操 作 接 口	功 能
data()	当前节点所存数据对象
pred()	当前节点前驱节点的位置
succ()	当前节点后继节点的位置
insertAsPred(e)	插入前驱节点，存入被引用对象e，返回新节点位置
insertAsSucc(e)	插入后继节点，存入被引用对象e，返回新节点位置

■ **ListNode模板类**

按照表3.1所定义的ADT接口，可定义列表节点模板类如代码3.1所示。出于简洁与效率的考虑，这里并未对**ListNode**对象做封装处理。列表节点数据项的类型，通过模板参数**T**指定。

```
1 typedef int Rank; //秩
2 #define ListNodePosi(T) ListNode<T>* //列表节点位置
3
4 template <typename T> struct ListNode { //列表节点模板类（以双向链表形式实现）
5 // 成员
6     T data; ListNodePosi(T) pred; ListNodePosi(T) succ; //数值、前驱、后继
7 // 构造函数
8     ListNode() {} //针对header和trailer的构造
9     ListNode( T e, ListNodePosi(T) p = NULL, ListNodePosi(T) s = NULL)
10         : data(e), pred(p), succ(s) {} //默认构造器
11 // 操作接口
12     ListNodePosi(T) insertAsPred(T const& e); //紧靠当前节点之前插入新节点
13     ListNodePosi(T) insertAsSucc(T const& e); //紧随当前节点之后插入新节点
14 };
```

代码3.1 列表节点模板类^①

每个节点都存有数据对象**data**。为保证叙述简洁，在不致歧义的前提下，本书将不再区分节点及其对应的**data**对象。此外，每个节点还设有指针**pred**和**succ**，分别指向其前驱和后继。

为了创建一个列表节点对象，只需根据所提供的参数，分别设置节点内部的各个变量。其中前驱、后继节点的位置指针若未予指定，则默认取作**NULL**。

3.2.2 列表

■ **ADT接口**

作为一种抽象数据类型，列表对象应支持以下操作接口。

表3.2 列表ADT支持的操作接口

操 作 接 口	功 能	适 用 对 象
size()	报告列表当前的规模（节点总数）	列表
first()、last()	返回首、末节点的位置	列表
insertAsFirst(e) insertAsLast(e)	将e当作首、末节点插入	列表
insertBefore(p, e) insertAfter(p, e)	将e当作节点p的直接前驱、后继插入	列表

^① 请注意，这里所“定义”的**ListNodePosi(T)**并非真正意义上“列表节点位置”类型。巧合的是，就在本书第1版即将付印之际，C++0x标准终于被ISO接纳。新标准所拓展的特性之一，就是对模板别名（**template alias**）等语法形式的支持。因此可以期望在不久的将来，C++编译器将能够支持如下更为直接和简明的描述和实现：

```
template <typename T> typedef ListNode<T>* ListNodePosi;
```

操 作 接 口	功 能	适 用 对 象
remove(p)	删除位置p处的节点，返回其数值	列表
disordered()	判断所有节点是否已按非降序排列	列表
sort()	调整各节点的位置，使之按非降序排列	列表
find(e)	查找目标元素e，失败时返回NULL	列表
search(e)	查找目标元素e，返回不大于e且秩最大的节点	有序列表
deduplicate()	剔除重复节点	列表
uniquify()	剔除重复节点	有序列表
traverse()	遍历并统一处理所有节点，处理方法由函数对象指定	列表

请留意用以指示插入和删除操作位置的节点p。这里约定，它或者在此前经查找已经确定，或者从此前的其它操作返回或沿用。这些也是列表类结构的典型操作方式。

这里也设置一个disordered()接口，以判断列表是否已经有序。同时，也分别针对有序和无序列表，提供了去重操作的两个版本（deduplicate()和uniquify()），以及查找操作的两个版本（find()和search()）。与向量一样，有序列表的唯一化，比无序列表效率更高。然而正如我们将要看到的，由于只能通过位置指针以局部移动的方式访问节点，尽管有序列表中节点在逻辑上始终按照大小次序排列，其查找操作的效率并没有实质改进（习题[3-1]）。

■ List模板类

按照表3.2定义的ADT接口，可定义List模板类如下。

```
1 #include "listNode.h" //引入列表节点类
2
3 template <typename T> class List { //列表模板类
4
5 private:
6     int _size; ListNodePosi(T) header; ListNodePosi(T) trailer; //规模、头哨兵、尾哨兵
7
8 protected:
9     void init(); //列表创建时的初始化
10    int clear(); //清除所有节点
11    void copyNodes(ListNodePosi(T), int); //复制列表中自位置p起的n项
12    void merge(ListNodePosi(T)&, int, List<T>&, ListNodePosi(T), int); //有序列表区间归并
13    void mergeSort(ListNodePosi(T)&, int); //对从p开始连续的n个节点归并排序
14    void selectionSort(ListNodePosi(T), int); //对从p开始连续的n个节点选择排序
15    void insertionSort(ListNodePosi(T), int); //对从p开始连续的n个节点插入排序
16
17 public:
18 // 构造函数
19     List() { init(); } //默认
20     List(List<T> const& L); //整体复制列表L
21     List(List<T> const& L, Rank r, int n); //复制列表L中自第r项起的n项
22     List(ListNodePosi(T) p, int n); //复制列表中自位置p起的n项
```

```

23 // 析构函数
24 ~List(); //释放 ( 包含头、尾哨兵在内的 ) 所有节点
25 // 只读访问接口
26 Rank size() const { return _size; } //规模
27 bool empty() const { return _size <= 0; } //判空
28 T& operator[](int r) const; //重载, 支持循秩访问 ( 效率低 )
29 ListNodePosi(T) first() const { return header->succ; } //首节点位置
30 ListNodePosi(T) last() const { return trailer->pred; } //末节点位置
31 bool valid(ListNodePosi(T) p) //判断位置p是否对外合法
32 { return p && (trailer != p) && (header != p); } //将头、尾节点等同于NULL
33 int disordered() const; //判断列表是否已排序
34 ListNodePosi(T) find(T const& e) const //无序列表查找
35 { return find(e, _size, trailer); }
36 ListNodePosi(T) find(T const& e, int n, ListNodePosi(T) p) const; //无序区间查找
37 ListNodePosi(T) search(T const& e) const //有序列表查找
38 { return search(e, _size, trailer); }
39 ListNodePosi(T) search(T const& e, int n, ListNodePosi(T) p) const; //有序区间查找
40 ListNodePosi(T) selectMax(ListNodePosi(T) p, int n); //在p及其前n-1个后继中选出最大者
41 ListNodePosi(T) selectMax() { return selectMax(header->succ, _size); } //整体最大者
42 // 可写访问接口
43 ListNodePosi(T) insertAsFirst(T const& e); //将e当作首节点插入
44 ListNodePosi(T) insertAsLast(T const& e); //将e当作末节点插入
45 ListNodePosi(T) insertBefore(ListNodePosi(T) p, T const& e); //将e当作p的前驱插入
46 ListNodePosi(T) insertAfter(ListNodePosi(T) p, T const& e); //将e当作p的后继插入
47 T remove(ListNodePosi(T) p); //删除合法位置p处的节点, 返回被删除节点
48 void merge(List<T>& L) { merge(first(), size, L, L.first(), L._size); } //全列表归并
49 void sort(ListNodePosi(T) p, int n); //列表区间排序
50 void sort() { sort(first(), _size); } //列表整体排序
51 int deduplicate(); //无序去重
52 int uniquify(); //有序去重
53 void reverse(); //前后倒置 ( 习题 )
54 // 遍历
55 void traverse(void (*)(T&)); //遍历, 依次实施visit操作 ( 函数指针, 只读或局部性修改 )
56 template <typename VST> //操作器
57 void traverse(VST&); //遍历, 依次实施visit操作 ( 函数对象, 可全局性修改 )
58 }; //List

```

代码3.2 列表模板类

由代码3.2可见, 列表结构的实现方式与第2章的向量结构颇为相似: 通过模板参数T指定列表元素的类型 (同时亦为代码3.1中列表节点数据项的类型); 在内部设置私有变量以记录当前规模等状态信息; 基于多种排序算法提供统一的sort()接口, 以将列表转化为有序列表。

以下, 分别介绍列表的内部结构、基本接口, 以及主要算法的具体实现。

§ 3.3 列表

3.3.1 头、尾节点

List对象的内部组成及逻辑结构如图3.1所示，其中私有的头节点（header）和尾节点（trailer）始终存在，但对外并不可见。对外部可见的数据节点如果存在，则其中的第一个和最后一个节点分别称作首节点（first node）和末节点（last node）。

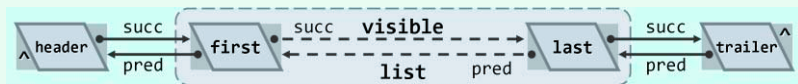


图3.1 首（末）节点是头（尾）节点的直接后继（前驱）

就内部结构而言，头节点紧邻于首节点之前，尾节点紧邻于末节点之后。这类经封装之后从外部不可见的节点，称作哨兵节点（sentinel node）。由代码3.2中List::valid()关于合法节点位置的判别准则可见，此处的两个哨兵节点从外部被等效地视作NULL。

设置哨兵节点之后，对于从外部可见的任一节点而言，其前驱和后继在列表内部都必然存在，故可简化算法的描述与实现。比如，在代码3.2中为实现first()和last()操作，只需直接返回header->succ或trailer->pred。此外更重要地，哨兵节点的引入，也使得相关算法不必再对各种边界退化情况做专门的处理，从而避免出错的可能，我们稍后将对此有更实际的体会。

尽管哨兵节点也需占用一定的空间，但只不过是常数规模，其成本远远低于由此带来的便利。

3.3.2 默认构造方法

创建List对象时，默认构造方法将调用如代码3.3所示的统一初始化过程init()，在列表内部创建一对头、尾哨兵节点，并适当地设置其前驱、后继指针构成一个双向链表。

```
1 template <typename T> void List<T>::init() { //列表初始化，在创建列表对象时统一调用
2     header = new ListNode<T>; //创建头哨兵节点
3     trailer = new ListNode<T>; //创建尾哨兵节点
4     header->succ = trailer; header->pred = NULL;
5     trailer->pred = header; trailer->succ = NULL;
6     _size = 0; //记录规模
7 }
```

代码3.3 列表类内部方法init()

如图3.2所示，该链表对外的有效部分初始为空，哨兵节点对外不可见，此后引入的新节点都将陆续插入于这一对哨兵节点之间。



图3.2 刚创建的List对象

在列表的其它构造方法中，内部变量的初始化过程与此相同，因此都可统一调用init()过程。该过程仅涉及常数次基本操作，共需运行常数时间。